

University of Tehran

Faculty of ECE



Digital systems design

CA4&5

Reza mohamadi

810103509

Spring 1405

Part 1a: Radix-4 Booth Multiplier Datapath Design

Overview of Datapath Schematic

The datapath for the 16-bit Radix-4 Booth Multiplier (RBM) is designed to process multiplication in 8 clock cycles by evaluating three bits of the multiplier at a time. The architecture consists of dedicated registers for operands, an Arithmetic Logic Unit (ALU) for partial product calculation, and a counter to track execution cycles.

To optimize the timing, the shift registers are designed such that the parallel inputs are connected via shifted wires, allowing both loading and shifting of 2 bits to occur within a single clock cycle.

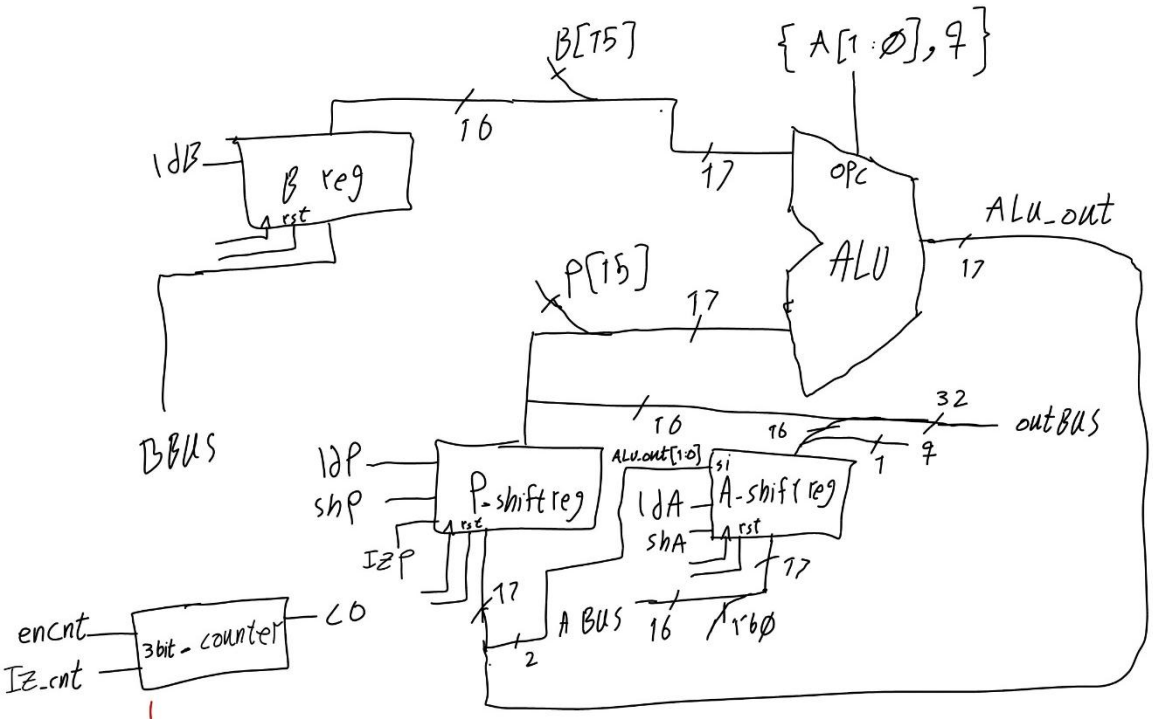
Hardware Components and Control Signals

Below is the detailed specification of each datapath component, including their functionalities and necessary control signals. All sequential components are synchronized to the global clock (clk) and can be reset via the global reset (rst) signal.

- **B Register (B reg)**
  - **Functionality:** A 16-bit register that stores the multiplicand received from the BBUS. Its output is sign-extended to 17 bits (B[15] appended) before feeding into the ALU.
  - **Control Signals:**
    - ldB (Load B): When high, loads the value from BBUS into the register.
- **A Shift Register (A-shift reg)**
  - **Functionality:** A shift register that holds the 16-bit multiplier A, alongside an extra bit q (acting as the implicit A_{-1} bit for the Booth algorithm). It shifts right by 2 bits per operation cycle. The lower 16 bits of the final 32-bit outBus are drawn from this register.
  - **Control Signals:**
    - ldA (Load A): When high, loads the multiplier from the ABUS.
    - shA (Shift A): When high, enables the 2-bit arithmetic right shift.
  - **Data Signals:** Takes a serial input (si) from the lower 2 bits of the ALU output (ALU_out[1:0]). It outputs {A[1:0], q} to the ALU to determine the current operation.
- **P Shift Register (P-shift reg)**
  - **Functionality:** A 17-bit shift register that accumulates the partial product (P). It is capable of loading the ALU output and shifting it right by 2 bits simultaneously. The upper 16 bits of the final 32-bit outBus are drawn from this register.
  - **Control Signals:**
    - ldP (Load P): When high, loads the calculated partial product from the ALU.
    - shP (Shift P): When high, enables the 2-bit right shift.
    - lzP (Load Zero P): Clears the register (sets to zero) at the start of a multiplication.
- **Cycle Counter (3-bit-counter)**

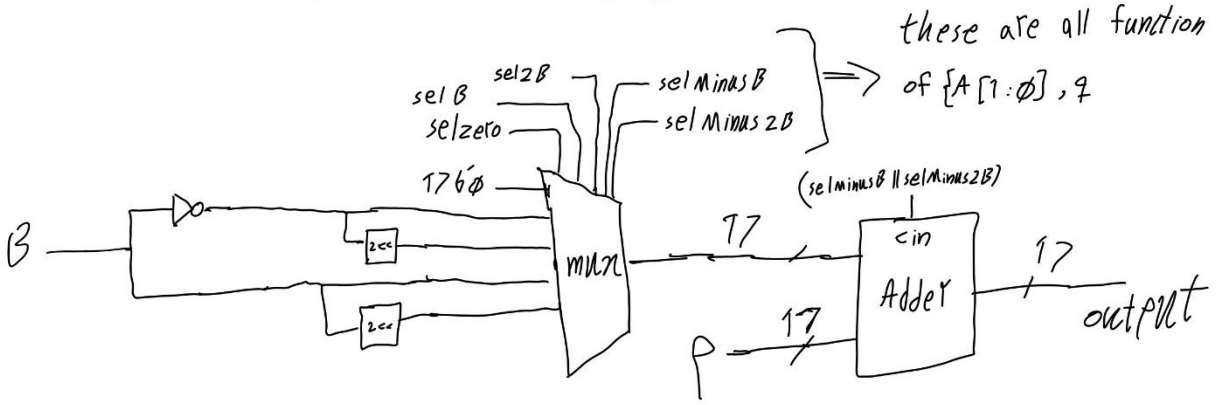
    - **Functionality:** A 3-bit counter used to track the number of shift/add operations. Since it is a 16-bit RBM, it requires $n/2 = 8$ cycles, making a 3-bit counter perfectly sized.
    - **Control Signals:**
        - encnt (Enable Count): Increments the counter each active cycle.
        - lz_cnt (Load Zero Counter): Resets the counter to 0 before a new multiplication begins.
    - **Data Signals:** Outputs a Carry Out (co) signal to the controller to indicate when the 8 cycles are complete.
- **Arithmetic Logic Unit (ALU)**
    - **Functionality:** A combinational block that computes the new partial product. Based on the {A[1:0], q} opcode, it generates $0, B, 2B, -B$, or $-2B$, and adds it to the current partial product P.
    - **Internal Architecture:**
        - **Operand B Modifiers:** Generates inverted B, left-shifted B $(2B)$, and inverted left-shifted B (for $-2B$).
        - **Multiplexer (MUX):** Selects the correct modified B operand using internal selector signals (selZero, selB, sel2B, selMinusB, selMinus2B). These selectors are direct combinational logic functions of {A[1:0], q}.
        - **17-bit Adder:** Adds the current P to the MUX output. For subtraction operations ($-B$ or $-2B$), the cin (carry-in) of the adder is driven high by the logic (selMinusB || selMinus2B) to complete the two's complement conversion. Outputs the 17-bit ALU_out.

Handwritten schematic of datapath:



the shift registers are designed to sift 2-bits when the shif enable input is high. also the parallel inputs are connected by shifted wires so bot Loading and then shifting can happen on a single clock cycle.

inside the ALU:



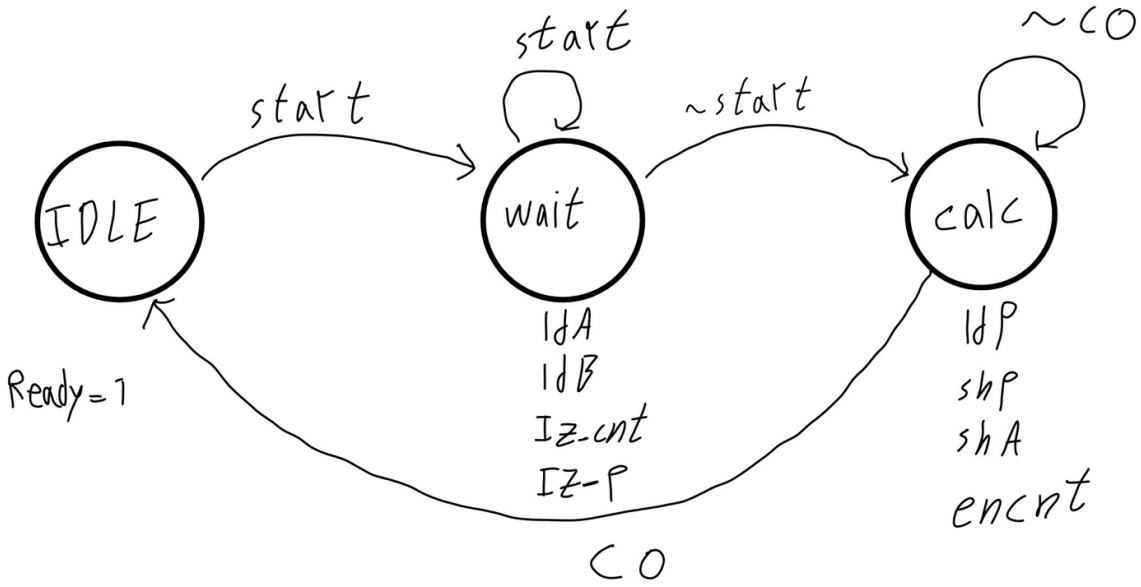
Part 1b: Controller State Diagram

Overview of the Controller The controller for the 16-bit sequential Radix-4 Booth Multiplier (RBM) is implemented as a Finite State Machine (FSM). It manages the initialization of the datapath registers and dictates the step-by-step execution of the multiplication process over the required clock cycles. The FSM consists of three primary states: `IDLE`, `wait`, and `calc`.

State Descriptions, Inputs, and Outputs The state transitions are governed by the `start` signal from the external environment and the carry-out (`CO`) signal from the datapath's 3-bit cycle counter. The outputs are exclusively state-dependent (Moore-type FSM).

- **State: IDLE**
  - **Description:** The system rests in this state when no multiplication is actively occurring. It signals to the external system that it is ready to accept a new operation.
  - **Outputs:** `Ready = 1`. All other control signals are de-asserted.
  - **Transitions:**
    - The FSM transitions to the `wait` state when the `start` signal is asserted (`start = 1`).
- **State: wait**
  - **Description:** This state handles the initialization phase. It waits for the `start` pulse to conclude before capturing the operands and resetting the datapath's accumulators and counters.
  - **Outputs:**
    - `ldA = 1`, `ldB = 1`: Loads the multiplier and multiplicand from the buses.
    - `lz_cnt = 1`: Clears the 3-bit cycle counter.
    - `lz_P = 1`: Initializes the partial product register to zero.
  - **Transitions:**
    - Remains in the `wait` state as long as the `start` signal is held high (`start`).
    - Transitions to the `calc` state on the first clock edge where the `start` signal is de-asserted (`~start`), triggering the beginning of the multiplication cycles.
- **State: calc**
  - **Description:** This state drives the core arithmetic and shifting operations of the Radix-4 Booth algorithm. It loops internally to perform the necessary additions/subtractions and 2-bit right shifts for each cycle.
  - **Outputs:**
    - `ldP = 1`: Loads the newly computed partial product from the ALU back into the P shift register.
    - `shP = 1`, `shA = 1`: Shifts both the P and A registers right by 2 bits.
    - `encnt = 1`: Enables the counter to increment, tracking the number of completed cycles.
  - **Transitions:**
    - Remains in the `calc` state as long as the counter's carry-out signal is zero (`~CO`), indicating the 8 cycles are not yet complete.
    - Transitions back to the `IDLE` state when the counter generates a carry-out (`CO = 1`), signaling that the multiplication has finished.

Controller state digram:



Part 1c: SystemVerilog Description of the Datapath

Overview This section provides the complete SystemVerilog structural description of the datapath for the 16-bit Radix-4 Booth Multiplier. The datapath is implemented using a modular approach, instantiating individual RTL components (registers, shifters, ALU, and counter) within a main datapath module.

Below are the SystemVerilog codes for the top-level datapath and its underlying sub-modules.

1. Main Datapath Integration (multiplierDP)

This module wires together all datapath components. It manages the sign-extension of the B and P registers to 17 bits before routing them to the ALU and handles the final concatenation of the P and A registers to form the 32-bit result.

```
module multiplierDP(
    input  logic clk, rst,
    input  logic [15:0] A, B,
    input  logic ldA, ldB, IzP, ldP, shA, shP, IzCnt, enCnt,
    output logic co,
    output logic [31:0] result
);

    logic [15:0] B_reg;
    logic [15:0] A_reg;
    logic q;
    logic [15:0] P_reg;
    logic [16:0] ALU_out;
    logic [16:0] sign_ext_B;
    logic [16:0] sign_ext_P;
```

```
    Bregister b_reg_inst (
        .clk(clk),
        .rst(rst),
        .ldB(ldB),
        .B_in(B),
        .B_out(B_reg)
    );

    assign sign_ext_B = {B_reg[15], B_reg};
    assign sign_ext_P = {P_reg[15], P_reg};

    multiplier_Alu alu_inst (
        .A(sign_ext_P),
        .B(sign_ext_B),
        .opc({A_reg[1:0], q}),
        .F(ALU_out)
    );

    PshiftReg p_reg_inst (
        .clk(clk),
        .rst(rst),
        .IzP(IzP),
        .ldP(ldP),
        .shP(shP),
        .P_in(ALU_out),
        .P_out(P_reg)
    );

    AshiftReg a_reg_inst (
        .clk(clk),
        .rst(rst),
        .ldA(ldA),
        .shA(shA),
        .A_in(A),
        .s_in(ALU_out[1:0]),
        .A_out(A_reg),
        .q(q)
    );

    counter cnt_inst (
        .clk(clk),
        .rst(rst),
        .IzCnt(IzCnt),
        .enCnt(enCnt),
        .co(co)
    );
    assign result = {P_reg, A_reg};
endmodule
```

2. Arithmetic Logic Unit (multiplier_Alu)

The combinational ALU evaluates the 3-bit opcode (opc) to determine the multiple of B to add to A. It handles the necessary shifting for $2B$ and two's complement inversion for subtraction operations.

```
module multiplier_Alu(
    input  logic [16:0] A,
    input  logic [16:0] B,
    input  logic [2:0]  opc,
    output logic [16:0] F
);

    logic [16:0] mux_out;
    logic cin;
    logic selZero, selB, sel2B, selMinusB, selMinus2B;

    assign selZero    = (opc == 3'b000) || (opc == 3'b111);
    assign selB       = (opc == 3'b001) || (opc == 3'b010);
    assign sel2B      = (opc == 3'b011);
    assign selMinus2B = (opc == 3'b100);
    assign selMinusB  = (opc == 3'b101) || (opc == 3'b110);

    assign cin = selMinusB || selMinus2B;

    always_comb begin
        if (selB)            mux_out = B;
        else if (sel2B)      mux_out = B << 1;
        else if (selMinusB)  mux_out = ~B;
        else if (selMinus2B) mux_out = ~(B << 1);
        else                 mux_out = 17'd0;
    end

    assign F = A + mux_out + {16'd0, cin};

endmodule
```

3. Shift Registers and Storage

These modules manage the operands and partial products. AshiftReg shifts the multiplier right by 2 bits per cycle. PshiftReg conditionally loads the ALU output and applies a 2-bit arithmetic right shift. Bregister provides static storage for the multiplicand.

```
module AshiftReg(
    input  logic clk, rst,
    input  logic ldA, shA,
    input  logic [15:0] A_in,
    input  logic [1:0]  s_in,
    output logic [15:0] A_out,
```

```
    output logic q
);

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            A_out <= 16'b0;
            q   <= 1'b0;
        end
        else if (ldA) begin
            A_out <= A_in;
            q   <= 1'b0;
        end
        else if (shA) begin
            A_out <= {s_in, A_out[15:2]};
            q   <= A_out[1];
        end
    end

endmodule
```

```
module Bregister(
    input  logic clk, rst,
    input  logic ldB,
    input  logic [15:0] B_in,
    output logic [15:0] B_out
);

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            B_out <= 16'b0;
        else if (ldB)
            B_out <= B_in;
    end

endmodule
```

```
module PshiftReg(
    input  logic clk,
    input  logic rst,
    input  logic IzP,
    input  logic ldP,
    input  logic shP,
    input  logic [16:0] P_in,
    output logic [15:0] P_out
);
```

```
    always_ff @(posedge clk or posedge rst)
    begin
        if (rst)
            P_out <= 16'd0;
        else if (IzP)
            P_out <= 16'd0;
        else if (ldP && shP)
            P_out <= {P_in[16], P_in[16], P_in[15:2]};
        else if (ldP)
            P_out <= P_in[15:0];
    end

endmodule
```

4. Cycle Counter (counter)

A 3-bit counter module that increments when enabled and asserts a carry-out (co) signal upon reaching a value of 7, signaling the completion of the 8 required operational cycles.

```
module counter(
    input  logic clk, rst,
    input  logic IzCnt, enCnt,
    output logic co
);

    logic [2:0] count;

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            count <= 3'b0;
        else if (IzCnt)
            count <= 3'b0;
        else if (enCnt)
            count <= count + 1'b1;
    end

    assign co = (count == 3'd7);

endmodule
```

Part 1d: Controller SystemVerilog Description

Overview This section provides the SystemVerilog implementation of the Radix-4 Booth Multiplier's controller. The module is constructed using the Huffman FSM modeling style, which strictly separates the sequential state-memory logic from the combinational next-state and output logic.

SystemVerilog Implementation

The multiplier_controller manages the multiplication sequence by transitioning between three states: idle, wait_st, and calc. The outputs are generated combinationally based solely on the current state (Moore model), and default assignments are initialized at the top of the always_comb block to prevent unintended latch inference.

```
module multiplier_controller(
    input logic clk, rst, start, co,
    output logic ldA, ldB, IzCnt, IzP, ldP, shP, shA,enCnt, ready
);

    localparam [1:0] idle    = 2'd0;
    localparam [1:0] wait_st = 2'd1;
    localparam [1:0] calc    = 2'd2;
    logic [1:0] ns, ps;

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            ps <= idle;
        else
            ps <= ns;
    end

    always_comb begin
        {ready, ldA, ldB, IzCnt, IzP, ldP, shP, shA,enCnt} = 9'b0;
        case (ps)
            idle: begin
                ns = start ? wait_st : idle;
                ready = 1'b1;
            end
            wait_st: begin
                ns = start ? wait_st : calc;
                {ldA, ldB, IzCnt, IzP} = 4'b1111;
            end
            calc: begin
                ns = co ? idle : calc;
                {ldP, shP, shA,enCnt} = 4'b1111;
            end
            default: ns = idle;
        endcase
    end
```

```
endmodule
```

Part 1e: Top-Level Module Architecture

Overview The top-level module serves as the wrapper that instantiates and connects the Datapath and the Controller. This architecture isolates the internal control and status signals from the external environment, exposing only the necessary interfaces to operate the multiplier.

External Signals

The external interface of the multiplier consists of the following wires and buses:

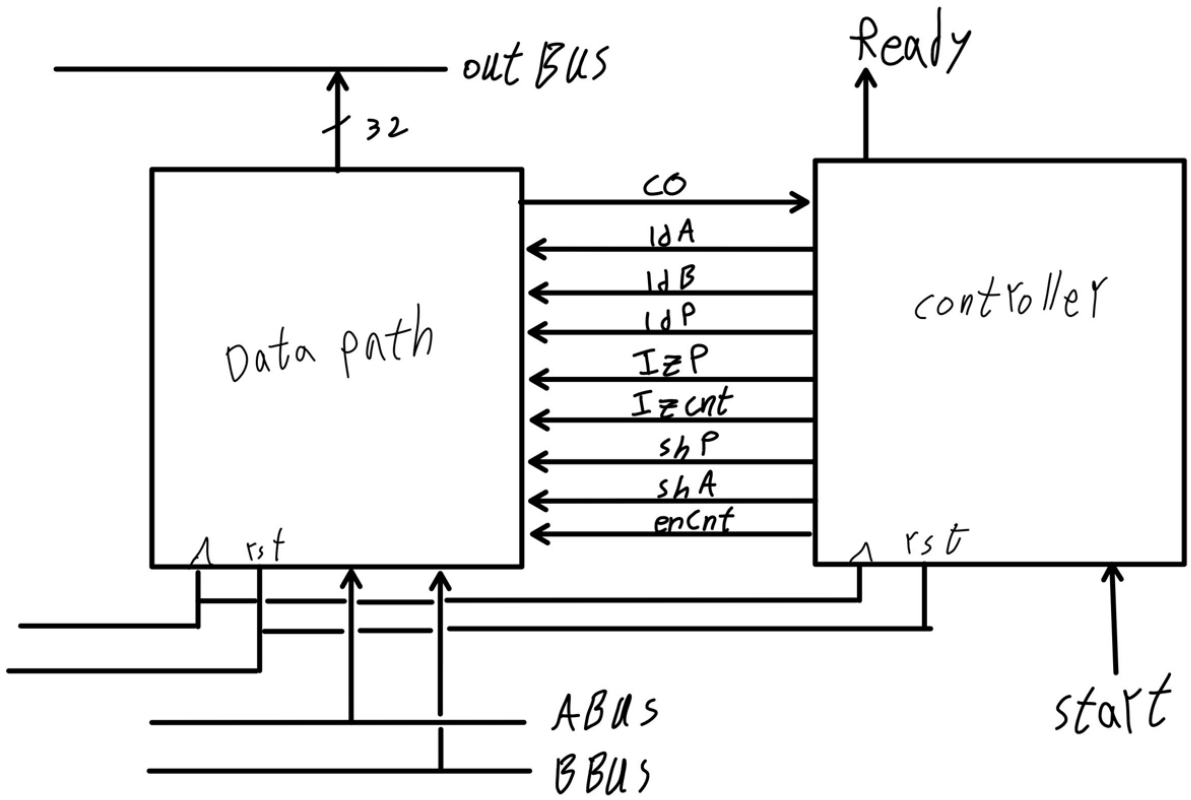
- **Inputs:**
  - clk, rst: Global clock and reset signals routed to both the datapath and the controller.
  - start: A 1-bit input signal to the controller to initiate the multiplication process.
  - ABUS, BBUS: 16-bit input buses providing the multiplier and multiplicand to the datapath.
- **Outputs:**
  - Ready: A 1-bit output from the controller indicating the system is idle and ready for a new operation.
  - outBus: A 32-bit output bus from the datapath carrying the final computed product.

Internal Connections

The datapath and controller communicate via a set of dedicated internal wires to execute the multiplication sequence:

- **Status Signal (Datapath to Controller):**
  - CO: The carry-out signal from the datapath's cycle counter, notifying the controller when the required cycles are complete.
- **Control Signals (Controller to Datapath):**
  - ldA, ldB: Load enable signals for the A (multiplier) and B (multiplicand) registers.
  - ldP: Load enable for the P (partial product) register.
  - lzP, lzcnt: Initialization (clear) signals for the partial product register and the cycle counter.
  - shA, shP: Shift enable signals to perform the 2-bit arithmetic right shifts.
  - enCnt: Enable signal to increment the cycle counter.

Handwritten diagram:



Part 1f: SystemVerilog Description of the Top-Level Module

Overview This section provides the complete SystemVerilog description of the top-level module for the Radix-4 Booth Multiplier. The radix4_multiplier_top module serves as the structural wrapper, putting together the datapath (multiplierDP) and the controller (multiplier_controller) and detailing how they are connected. It isolates the internal state and control logic, exposing only the external wires and buses required for the multiplier's operation.

SystemVerilog Implementation The code below instantiates the datapath and controller, routing the global clk and rst to both components. The external 16-bit ABUS and BBUS are routed directly to the datapath, while the start signal is routed to the controller. Internal control signals and the co status flag are wired seamlessly between the two sub-modules.

```
module radix4_multiplier_top (
    input  logic clk, rst, start,
    input  logic [15:0] ABUS, BBUS,
    output logic ready,
    output logic [31:0] outBUS
);

    logic co;
    logic ldA, ldB, ldP, IzP, IzCnt, shP, shA, enCnt;
```

```
    multiplierDP dp_inst (
        .clk(clk),
        .rst(rst),
        .A(ABUS),
        .B(BBUS),
        .ldA(ldA),
        .ldB(ldB),
        .IzP(IzP),
        .ldP(ldP),
        .shA(shA),
        .shP(shP),
        .IzCnt(IzCnt),
        .enCnt(enCnt),
        .co(co),
        .result(outBUS)
    );

    multiplier_controller ctrl_inst (
        .clk(clk),
        .rst(rst),
        .start(start),
        .co(co),
        .ready(ready),
        .ldA(ldA),
        .ldB(ldB),
        .ldP(ldP),
        .IzP(IzP),
        .IzCnt(IzCnt),
        .shP(shP),
        .shA(shA),
        .enCnt(enCnt)
    );

endmodule
```

Simulation Part B: Datapath Independent Testing

Overview To verify the correct operation of the arithmetic and shifting logic, the datapath of the Radix-4 Booth Multiplier was simulated and tested independently of the control unit.

Testbench Implementation

A dedicated SystemVerilog testbench (tb_datapath) was developed to stimulate the multiplierDP module. Because the controller is absent in this test, the testbench manually sequences the control signals (such as ldA, ldB, shP, shA, and enCnt) to replicate the exact timing and execution phases required by the RBM algorithm.

The testbench is programmed to evaluate a specific signed multiplication case: A = 5 and B = -3. It loads the operands, cycles the clock 8 times to represent the $n/2$ shifts, and uses an automated check to verify if the 32-bit output matches the expected product of -15.

```
`timescale 1ns/1ns

module tb_datapath;

    logic clk;
    logic rst;
    logic [15:0] A;
    logic [15:0] B;
    logic ldA;
    logic ldB;
    logic IzP;
    logic ldP;
    logic shA;
    logic shP;
    logic IzCnt;
    logic enCnt;

    logic co;
    logic [31:0] result;

    multiplierDP uut (
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .ldA(ldA),
        .ldB(ldB),
        .IzP(IzP),
        .ldP(ldP),
        .shA(shA),
        .shP(shP),
        .IzCnt(IzCnt),
        .enCnt(enCnt),
        .co(co),
        .result(result)
    );

    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    initial begin
        rst = 0;
        A = 16'd0;
```

```
        B = 16'd0;
        ldA = 0;
        ldB = 0;
        IzP = 0;
        ldP = 0;
        shA = 0;
        shP = 0;
        IzCnt = 0;
        enCnt = 0;

        @(negedge clk);
        rst = 1;
        @(negedge clk);
        rst = 0;

        @(negedge clk);
        A = 16'sd5;
        B = -16'sd3;
        ldA = 1;
        ldB = 1;
        IzP = 1;
        IzCnt = 1;

        @(negedge clk);

        ldA = 0;
        ldB = 0;
        IzP = 0;
        IzCnt = 0;

        ldP = 1;
        shP = 1;
        shA = 1;
        enCnt = 1;

        repeat(8) @(negedge clk);

        ldP = 0;
        shP = 0;
        shA = 0;
        enCnt = 0;

        @(negedge clk);

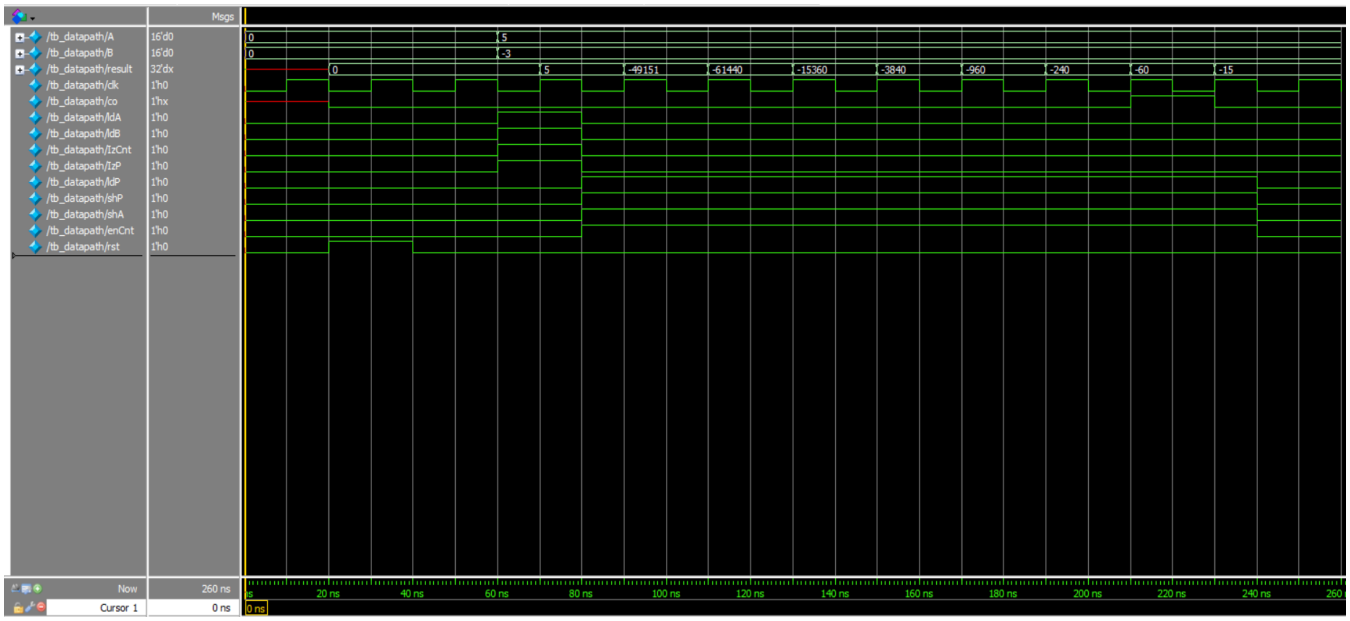
        if ($signed(result) == -32'sd15) begin
            $display("Test Passed: 5 * -3 = %0d", $signed(result));
        end else begin
            $display("Test Failed: 5 * -3 != %0d, expected -15",
$signed(result));
```

```
        end

        $stop;
    end
endmodule
```

Simulation Results

The independent datapath simulation successfully executed the signed multiplication.



- **Waveform Analysis:** The generated waveform demonstrates the proper loading of the 5 and -3 operands. Following the de-assertion of the load signals, the result bus correctly updates across the subsequent 8 clock cycles as the partial products are evaluated and shifted.

```
VSIM 3> run -all
# Test Passed: 5 * -3 = -15
# ** Note: $stop    : tb_datapath.sv(96)
#    Time: 260 ns  Iteration: 1  Instance: /tb_datapath
```

- **Console Verification:** The testbench transcript confirmed mathematical accuracy, outputting Test Passed: 5 * -3 = -15 at 260 ns and halting execution without errors.

Simulation Part C: Controller Independent Testing

Overview To ensure the correct sequencing of the multiplication algorithm, the control unit of the multiplier circuit was simulated and tested independently of the datapath. This verifies that the Finite State Machine (FSM) accurately transitions between states and asserts the correct control signals at each phase.

Testbench Implementation

A dedicated testbench (tb_controller) was written to isolate the multiplier_controller module. Because the datapath is not connected, the testbench manually stimulates the controller using the external start signal and mocks the datapath's carry-out (co) signal.

The testbench initializes the module, asserts the start signal to trigger the transition out of the idle state, waits for 8 clock cycles (representing the calculation phase), and then asserts the co signal to force the controller back to the idle state.

```
`timescale 1ns/1ns

module tb_controller;

  logic clk;
  logic rst;
  logic ldA;
  logic ldB;
  logic IzP;
  logic ldP;
  logic shA;
  logic shP;
  logic IzCnt;
  logic enCnt;
  logic co;
  logic start;
  logic ready;

  multiplier_controller uut(
                          .clk(clk),
                          .rst(rst),
                          .start(start),
                          .co(co),
                          .ldA(ldA),
                          .ldB(ldB),
                          .IzCnt(IzCnt),
                          .IzP(IzP),
                          .ldP(ldP),
                          .shP(shP),
                          .shA(shA),
                          .enCnt(enCnt),
                          .ready(ready)
                        );

  initial
  begin
    clk = 0;
    forever
      #10 clk = ~clk;
  end
```

```
  initial
  begin
    rst = 0;
    start = 0;
    co = 0;

    @(negedge clk);
    rst = 1;
    @(negedge clk);
    rst = 0;

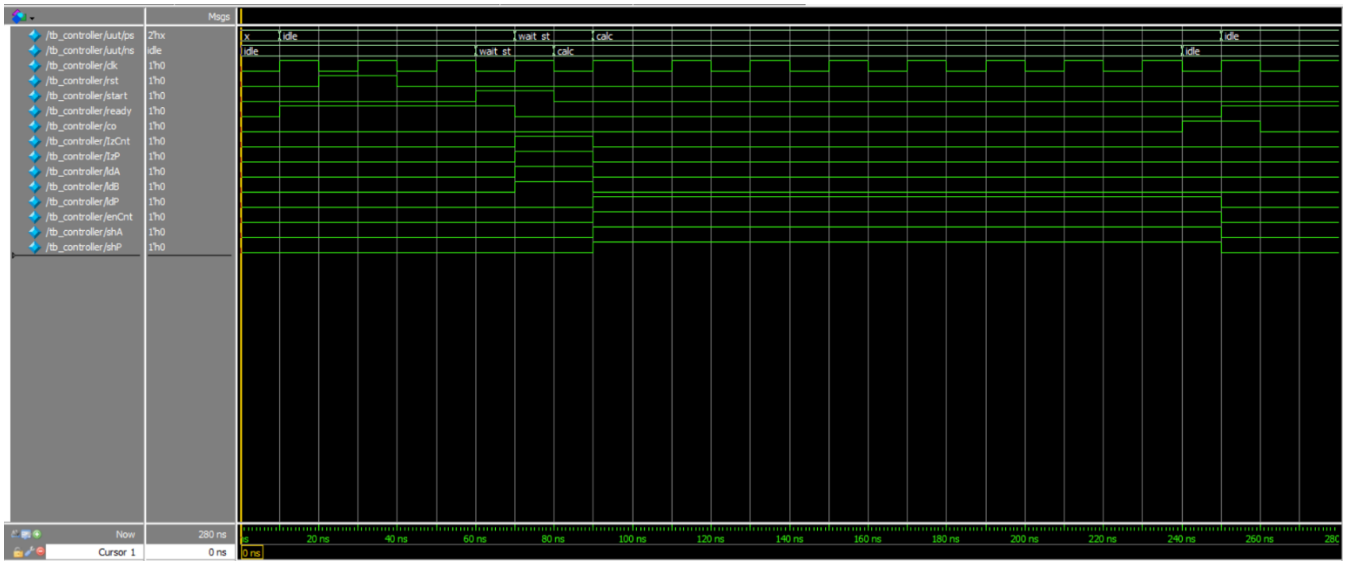
    @(negedge clk);
    start = 1;
    @(negedge clk);
    start = 0;

    repeat(8) @(negedge clk);

    co = 1;
    @(negedge clk);
    co = 0;
    @(negedge clk);
    $stop;
  end
endmodule
```

Simulation Results

The independent controller simulation successfully verified the state machine logic.



- **Waveform Analysis:** The generated waveform clearly illustrates the FSM behavior.
    - During the idle state, the ready signal is asserted.
    - Upon detecting start = 1, the next state (ns) transitions to wait_st, asserting the initialization signals (ldA, ldB, lzCnt, lzP).
    - As start drops to 0, the FSM moves to the calc state, properly asserting the execution signals (ldP, enCnt, shA, shP) for the required 8 cycles.
    - Finally, the simulated assertion of the co signal successfully returns the FSM to the idle state, bringing ready back high.

Simulation Part D: Top-Level Multiplier Testing

Overview To finalize the functional verification, the datapath and control unit were wired together within the top-level module and tested as a complete, integrated circuit. This ensures that the controller accurately drives the datapath in response to external signals and that the datapath computes the correct final product over the 8-cycle sequence.

Testbench Implementation

A comprehensive SystemVerilog testbench (tb_top) was developed to rigorously evaluate the radix4_multiplier_top module. The testbench utilizes an automated task (run_test) to apply various combinations of 16-bit signed operands to ABUS and BBUS.

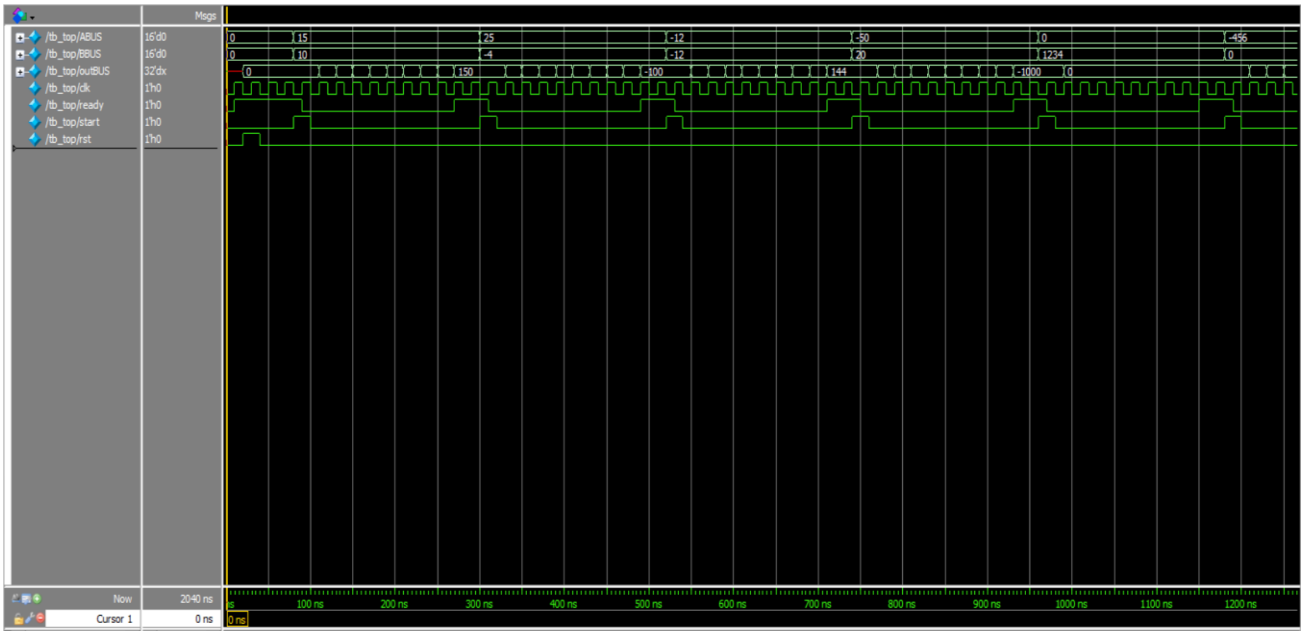
For each test case, the testbench asserts the start signal, waits for the controller to pull the ready signal low during calculation, and then waits for ready to return high. Once the multiplication is complete, it automatically compares the 32-bit outBUS against the expected mathematical result, incrementing pass/fail counters accordingly.

Simulation Results

The integrated testbench executed a total of 9 distinct multiplication scenarios to thoroughly verify the algorithm's robustness. These cases covered positive by positive, positive by negative, negative by negative, multiplications involving zero, and boundary values (e.g., maximum positive and minimum negative 16-bit integers).

```
# Radix-4 Booth Multiplier Testbench
# ==================================
#
# --- test 1:
# ABUS =     15 | BBUS =     10 | outBUS =         150 (Expected =         150)  [PASS]
#
# --- test 2:
# ABUS =     25 | BBUS =     -4 | outBUS =        -100 (Expected =        -100)  [PASS]
#
# --- test 3:
# ABUS =    -12 | BBUS =    -12 | outBUS =         144 (Expected =         144)  [PASS]
#
# --- test 4:
# ABUS =    -50 | BBUS =     20 | outBUS =       -1000 (Expected =       -1000)  [PASS]
#
# --- test 5:
# ABUS =      0 | BBUS =   1234 | outBUS =           0 (Expected =           0)  [PASS]
# ABUS =   -456 | BBUS =      0 | outBUS =           0 (Expected =           0)  [PASS]
#
# --- test 6:
# ABUS =      1 | BBUS = -32768 | outBUS =      -32768 (Expected =      -32768)  [PASS]
# ABUS =  32767 | BBUS =      2 | outBUS =       65534 (Expected =       65534)  [PASS]
# ABUS =     -1 | BBUS =     -1 | outBUS =           1 (Expected =           1)  [PASS]
#
#  tets results summery :
#
#  Total Tests Executed : 9
#  Passed               : 9
#  Failed               : 0
# ** Note: $stop    : tb_top.sv(102)
#    Time: 2040 ns  Iteration: 1  Instance: /tb_top
```

- **Console Verification:** As shown in the transcript output all 9 test cases evaluated successfully, yielding a 100% pass rate.



- **Waveform Analysis:** The generated waveform visually confirms the correct timing and handshaking. The ready signal correctly toggles in response to start, and the sequential state transitions process the data perfectly, ensuring outBUS contains the valid 32-bit product upon the rising edge of ready.

ASIC Synthesis: 45nm Library Target

Overview The complete top-level multiplier module (radix4_multiplier_top) was synthesized using Yosys targeting a standard 45nm technology library. The entire design synthesizes into a total of **410 standard cells** across **371 wires** (779 wire bits).

Structural Wrappers and Design Hierarchy The top-level files act as structural containers rather than containing direct logic gates:

- **Top-Level Module (radix4_multiplier_top):** The highest level of the hierarchy contains **2 macro-cells** (representing the instantiated datapath and controller). It utilizes 16 wires (77 wire bits) to route external inputs and connect the two main subsystems.

```
=== radix4_multiplier_top ===

   Number of wires:                 16
   Number of wire bits:             77
   Number of public wires:          16
   Number of public wire bits:      77
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                  2
     multiplierDP                    1
     multiplier_controller           1
```

- **Datapath Wrapper (multiplierDP):** This container holds **5 macro-cells** (the instantiated AshiftReg, Bregister, PshiftReg, counter, and multiplier_Alu). It utilizes 21 wires (175 wire bits) for internal datapath routing.

```
=== multiplierDP ===

   Number of wires:                 21
   Number of wire bits:            175
   Number of public wires:          21
   Number of public wire bits:     175
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                  5
     AshiftReg                       1
     Bregister                       1
     PshiftReg                       1
     counter                         1
     multiplier_Alu                  1
```

- **Flattened Design Hierarchy:** When fully flattened, the entire design synthesizes into a total of **410 standard cells** across **371 wires** (779 wire bits).

```
=== design hierarchy ===

   radix4_multiplier_top            1
     multiplierDP                   1
       AshiftReg                    1
       Bregister                    1
       PshiftReg                    1
       counter                      1
       multiplier_Alu               1
     multiplier_controller          1

   Number of wires:               371
   Number of wire bits:           779
   Number of public wires:         81
   Number of public wire bits:    442
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:               410
     $_AND_                        23
     $_AOI3_                       27
     $_DFF_PP0_                    54
     $_DFF_PP1_                     1
     $_MUX_                       118
     $_NAND_                       12
     $_NOR_                        50
     $_NOT_                        52
     $_OAI3_                       15
     $_OAI4_                        1
     $_OR_                         18
     $_XNOR_                       32
     $_XOR_                         7
```

Module Cell Breakdown

The distribution and types of standard cells across the sub-modules are detailed below:

- **ALU (multiplier_Alu):** This combinational module has the highest cell count in the design at **234 cells**. The composition includes:

```
=== multiplier_Alu ===

   Number of wires:                221
   Number of wire bits:            271
   Number of public wires:           4
   Number of public wire bits:      54
   Number of memories:               0
   Number of memory bits:            0
   Number of processes:              0
   Number of cells:                234
     $_AND_                          3
     $_AOI3_                        27
     $_MUX_                         35
     $_NAND_                        10
     $_NOR_                         47
     $_NOT_                         45
     $_OAI3_                        14
     $_OR_                          17
     $_XNOR_                        29
     $_XOR_                          7
```

- **Shift Register P (PshiftReg):**

```
=== PshiftReg ===

   Number of wires:                42
   Number of wire bits:            88
   Number of public wires:          7
   Number of public wire bits:     38
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                66
     $_AND_                        16
     $_DFF_PP0_                    16
     $_MUX_                        32
     $_NAND_                        1
     $_NOT_                         1
```

- **Shift Register A (AshiftReg):**

```
=== AshiftReg ===

   Number of wires:                28
   Number of wire bits:            74
   Number of public wires:          8
   Number of public wire bits:     39
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                52
     $_AND_                         1
     $_DFF_PP0_                    17
     $_MUX_                        33
     $_NOT_                         1
```

- **Operand B Register (Bregister):**

```
=== Bregister ===

   Number of wires:                 6
   Number of wire bits:            51
   Number of public wires:          5
   Number of public wire bits:     35
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                32
     $_DFF_PP0_                    16
     $_MUX_                        16
```

- **Cycle Counter (counter):**

```
=== counter ===

   Number of wires:                15
   Number of wire bits:            19
   Number of public wires:          6
   Number of public wire bits:      8
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                15
     $_AND_                         2
     $_DFF_PP0_                     3
     $_MUX_                         2
     $_NOR_                         3
     $_NOT_                         2
     $_XNOR_                        3
```

- **Controller (multiplier_controller):**

```
=== multiplier_controller ===

   Number of wires:                22
   Number of wire bits:            24
   Number of public wires:         14
   Number of public wire bits:     16
   Number of memories:              0
   Number of memory bits:           0
   Number of processes:             0
   Number of cells:                11
     $_AND_                         1
     $_DFF_PP0_                     2
     $_DFF_PP1_                     1
     $_NAND_                        1
     $_NOT_                         3
     $_OAI3_                        1
     $_OAI4_                        1
     $_OR_                          1
```

ASIC Synthesis Part B: Post-Synthesis Simulation Verification

Overview To verify that the gate-level netlist generated by Yosys synthesis using the 45nm standard cell library functions identically to the pre-synthesis SystemVerilog RTL description, a post-synthesis simulation was conducted. The goal of this verification step is to ensure no functional discrepancies, logical errors, or unintended behaviors were introduced during technology mapping and gate substitution.

Post-Synthesis Testbench Implementation

The post-synthesis testbench (tb_synth_radix4_multiplier_top) instantiates the synthesized top-level gate-level module (radix4_multiplier_top). It applies the exact same set of 9 test vectors previously evaluated during RTL behavioral simulation, testing positive numbers, negative numbers (two's complement), zero products, and edge conditions across the 16-bit signed operand space.

```
`timescale 1ns/1ns

module tb_synth_radix4_multiplier_top;


    logic clk;
    logic rst;
    logic start;
    logic signed [15:0] ABUS;
    logic signed [15:0] BBUS;
    logic ready;
    logic signed [31:0] outBUS;

    logic signed [31:0] expected_out;
    int pass_count = 0;
    int fail_count = 0;

    radix4_multiplier_top uut (
        .clk(clk),
        .rst(rst),
        .start(start),
        .ABUS(ABUS),
        .BBUS(BBUS),
        .ready(ready),
        .outBUS(outBUS)
    );

    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    task run_test(input logic signed [15:0] a_val, input logic signed [15:0]
b_val);
        begin
            @(negedge clk);
            ABUS = a_val;
            BBUS = b_val;
            expected_out = a_val * b_val;

            start = 1'b1;
            @(negedge clk);
```

```
            start = 1'b0;

            wait(ready == 1'b0);
            wait(ready == 1'b1);
            @(negedge clk);

            if (outBUS === expected_out) begin
                $display("ABUS = %6d | BBUS = %6d | outBUS = %11d (Expected =
%11d)  [PASS]",
                         a_val, b_val, outBUS, expected_out);
                pass_count++;
            end else begin
                $display("ABUS = %6d | BBUS = %6d | outBUS = %11d (Expected =
%11d)  [FAIL]",
                         a_val, b_val, outBUS, expected_out);
                fail_count++;
            end
        end
    endtask

    initial begin
        rst   = 0;
        start = 0;
        ABUS  = 16'd0;
        BBUS  = 16'd0;

        $display("Radix-4 Booth Multiplier Post-Synthesis Testbench");
        $display("================================================");

        @(negedge clk);
        rst = 1'b1;
        @(negedge clk);
        rst = 1'b0;
        @(negedge clk);

        wait(ready == 1'b1);

        $display("\n--- test 1: ");
        run_test(16'sd15, 16'sd10);

        $display("\n--- test 2:");
        run_test(16'sd25, -16'sd4);

        $display("\n--- test 3:");
        run_test(-16'sd12, -16'sd12);

        $display("\n--- test 4:");
        run_test(-16'sd50, 16'sd20);
```

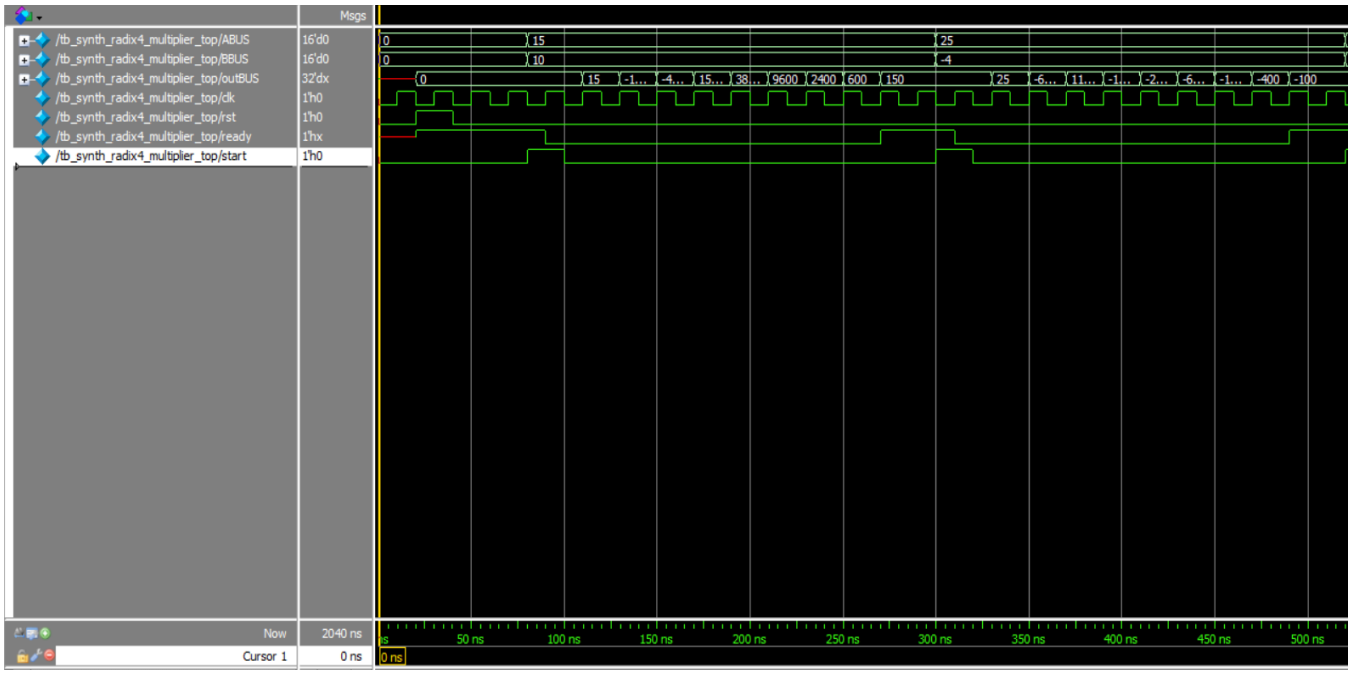
```
        $display("\n--- test 5:");
        run_test(16'sd0, 16'sd1234);
        run_test(-16'sd456, 16'sd0);

        $display("\n--- test 6:");
        run_test(16'sd1, -16'sd32768);
        run_test(16'sd32767, 16'sd2);
        run_test(-16'sd1, -16'sd1);

        $display("\n test results summary : \n");
        $display(" Total Tests Executed : %0d", (pass_count + fail_count));
        $display(" Passed               : %0d", pass_count);
        $display(" Failed               : %0d", fail_count);
        $stop;
    end

endmodule
```

Post-Synthesis Simulation Results & Comparison The post-synthesis functional verification confirmed complete equivalence with the pre-synthesis design:



- **Waveform Equivalence:** The post-synthesis waveform snippet demonstrates identical operational timing. The state machine correctly transitions through initialization and computation states, and the partial product shifts execute over 8 clock cycles before asserting ready.

```
# Radix-4 Booth Multiplier Post-Synthesis Testbench
# ================================================
#
# --- test 1:
# ABUS =     15 | BBUS =     10 | outBUS =         150 (Expected =         150)  [PASS]
#
# --- test 2:
# ABUS =     25 | BBUS =     -4 | outBUS =        -100 (Expected =        -100)  [PASS]
#
# --- test 3:
# ABUS =    -12 | BBUS =    -12 | outBUS =         144 (Expected =         144)  [PASS]
#
# --- test 4:
# ABUS =    -50 | BBUS =     20 | outBUS =       -1000 (Expected =       -1000)  [PASS]
#
# --- test 5:
# ABUS =      0 | BBUS =   1234 | outBUS =           0 (Expected =           0)  [PASS]
# ABUS =   -456 | BBUS =      0 | outBUS =           0 (Expected =           0)  [PASS]
#
# --- test 6:
# ABUS =      1 | BBUS = -32768 | outBUS =      -32768 (Expected =      -32768)  [PASS]
# ABUS =  32767 | BBUS =      2 | outBUS =       65534 (Expected =       65534)  [PASS]
# ABUS =     -1 | BBUS =     -1 | outBUS =           1 (Expected =           1)  [PASS]
#
#  test results summary :
#
#  Total Tests Executed : 9
#  Passed               : 9
#  Failed               : 0
# ** Note: $stop    : tb_synth_radix4_multiplier_top.sv(102)
#    Time: 2040 ns  Iteration: 1  Instance: /tb_synth_radix4_multiplier_top
```

- **Verification Summary:** As captured in the simulation transcript all 9 test cases passed without failure (0 errors). This confirms that the logic synthesis performed by Yosys using the 45nm library preserved 100% functional correctness.

FPGA Synthesis Part A: Quartus Compilation and Symbol Generation

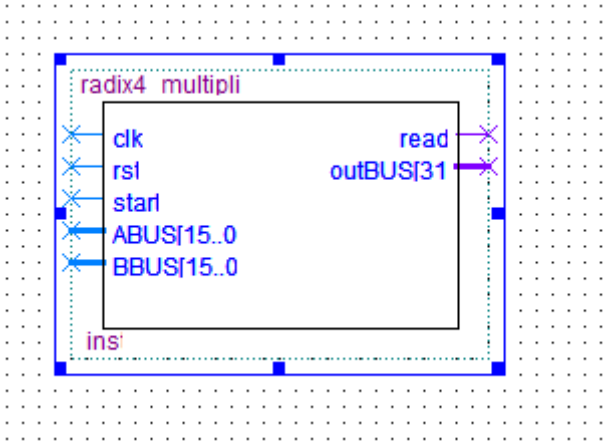
Overview The complete top-level Radix-4 Booth Multiplier module (radix4_multiplier_top) was imported into Intel Quartus Prime, targeting the Cyclone IV E FPGA family. The design underwent analysis and synthesis to verify structural integrity and create a schematic block symbol for instantiation within top-level Quartus Block Diagram Files (.bdf).

Synthesis Workflow & Symbol Generation

1. **Target Architecture Selection:** The Quartus project was configured targeting a Cyclone IV E device
2. **Analysis & Logic Synthesis:** All SystemVerilog design files (radix4_multiplier_top, multiplierDP, multiplier_controller, ALU, shift registers, and counter) were synthesized to perform RTL parsing, elaboration, and technology mapping for the Cyclone IV E architecture.
3. **Block Symbol File (.bsf) Generation:** Using the **Create/Update → Create Symbol Files for Current File** feature in Quartus, a modular schematic symbol was synthesized directly from the top-level entity.

Symbol Interface Definition As captured in the generated block symbol the module encapsulates all internal datapath and state machine wires, exposing the required external interface:

- **Input Ports (Left Side):**
  - clk: Global clock signal.
  - rst: Asynchronous global reset signal.
  - start: Execution start pulse signal.
  - ABUS[15..0]: 16-bit input bus for Operand A (multiplier).
  - BBUS[15..0]: 16-bit input bus for Operand B (multiplicand).
- **Output Ports (Right Side):**
  - ready: Handshaking output signal indicating when the operation completes.
  - outBUS[31..0]: 32-bit output bus driving the computed product.



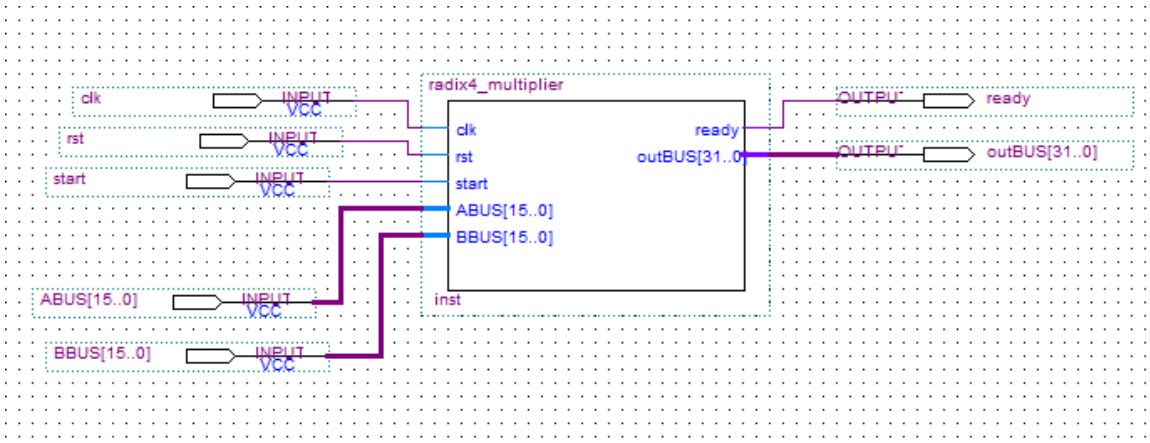
FPGA Synthesis Part B: Top-Level Block Diagram Instantiation

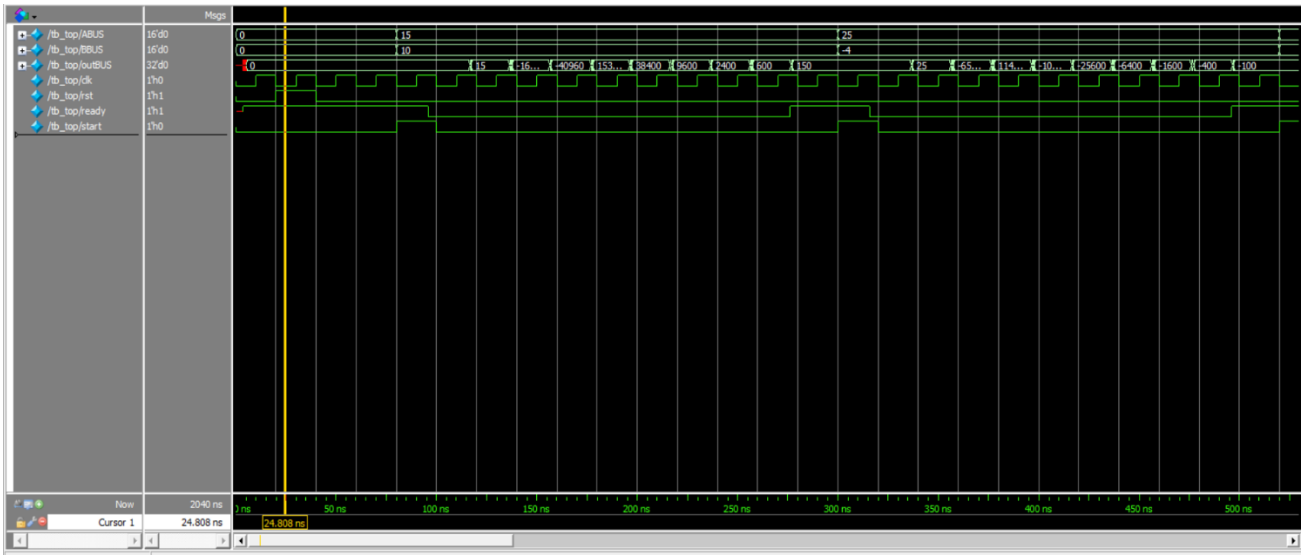
Overview Following the successful generation of the modular symbol, the multiplier was instantiated within a top-level Quartus Block Diagram File (.bdf). This schematic-based approach visually routes the external FPGA pins to the internal multiplier logic, establishing the final physical interface for the Cyclone IV E target.

Schematic Implementation and Pin Connections

As illustrated in the provided block diagram ,the radix4_multiplier symbol is placed at the center of the design and wired to standard Quartus input and output pin primitives.

- **Input Connections:**
  - Dedicated single-bit input pins are routed to the clk, rst, and start ports.
  - 16-bit input bus pins are connected to the ABUS[15..0] and BBUS[15..0] ports to supply the multiplier and multiplicand operands.
- **Output Connections:**
  - A single-bit output pin is wired to the ready port to flag the completion of the multiplication cycle.
  - A 32-bit output bus pin is routed from the outBUS[31..0] port to externalize the final computed product.






```
# Radix-4 Booth Multiplier: Post-Synthesis Gate-Level Testbench
# ============================================================
#
# --- test 1:
# ABUS =     15 | BBUS =     10 | outBUS =         150 (Expected =         150)  [PASS]
#
# --- test 2:
# ABUS =     25 | BBUS =     -4 | outBUS =        -100 (Expected =        -100)  [PASS]
#
# --- test 3:
# ABUS =    -12 | BBUS =    -12 | outBUS =         144 (Expected =         144)  [PASS]
#
# --- test 4:
# ABUS =    -50 | BBUS =     20 | outBUS =       -1000 (Expected =       -1000)  [PASS]
#
# --- test 5:
# ABUS =      0 | BBUS =   1234 | outBUS =           0 (Expected =           0)  [PASS]
# ABUS =   -456 | BBUS =      0 | outBUS =           0 (Expected =           0)  [PASS]
#
# --- test 6:
# ABUS =      1 | BBUS = -32768 | outBUS =      -32768 (Expected =      -32768)  [PASS]
# ABUS =  32767 | BBUS =      2 | outBUS =       65534 (Expected =       65534)  [PASS]
# ABUS =     -1 | BBUS =     -1 | outBUS =           1 (Expected =           1)  [PASS]
#
#  test results summary :
#
#  Total Tests Executed : 9
#  Passed               : 9
#  Failed               : 0
# ** Note: $stop    : tb_top_postFpga_synth.sv(105)
#    Time: 2040 ns  Iteration: 1  Instance: /tb_top
```

FPGA Synthesis Part C: Generation of .vo and .sdo Files

Overview Following the schematic instantiation, the complete Quartus Block Diagram was fully compiled and synthesized for the Cyclone IV E target. The EDA Netlist Writer tool in Quartus was then executed to generate the files required for gate-level timing simulation:

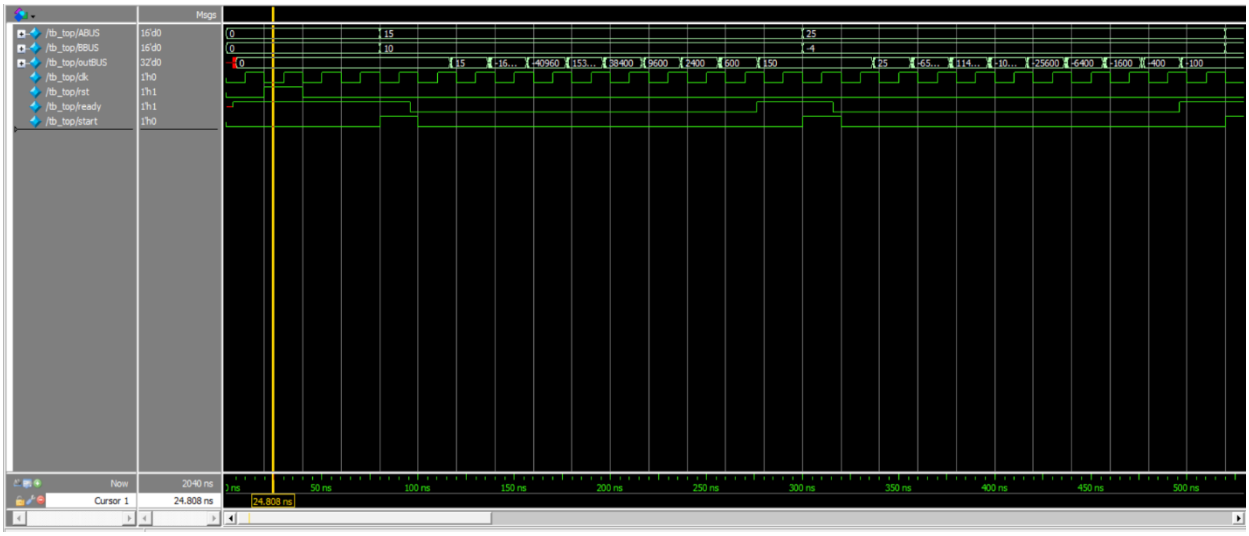
- **Verilog Output File (.vo):** Contains the flattened, gate-level netlist where all RTL logic has been mapped to physical Cyclone IV E hardware primitives (such as Lookup Tables or LUTs).
- **Standard Delay Output File (.sdo):** Contains the physical timing parameters, representing the real-world delays of the synthesized logic gates and the copper routing between them.

FPGA Synthesis Part D: Post-Synthesis Gate-Level Simulation

Overview To perform the post-synthesis simulation, the original functional testbench was updated to instantiate the gate-level .vo file and back-annotate the .sdo timing file. The testbench was then executed using the exact same 9 test vectors to directly compare the post-synthesis results against the initial RTL simulation.

Simulation Results

```
# Radix-4 Booth Multiplier: Post-Synthesis Gate-Level Testbench
# ============================================================
#
# --- test 1:
# ABUS =     15 | BBUS =     10 | outBUS =         150 (Expected =         150)  [PASS]
#
# --- test 2:
# ABUS =     25 | BBUS =     -4 | outBUS =        -100 (Expected =        -100)  [PASS]
#
# --- test 3:
# ABUS =    -12 | BBUS =    -12 | outBUS =         144 (Expected =         144)  [PASS]
#
# --- test 4:
# ABUS =    -50 | BBUS =     20 | outBUS =       -1000 (Expected =       -1000)  [PASS]
#
# --- test 5:
# ABUS =      0 | BBUS =   1234 | outBUS =           0 (Expected =           0)  [PASS]
# ABUS =   -456 | BBUS =      0 | outBUS =           0 (Expected =           0)  [PASS]
#
# --- test 6:
# ABUS =      1 | BBUS = -32768 | outBUS =      -32768 (Expected =      -32768)  [PASS]
# ABUS =  32767 | BBUS =      2 | outBUS =       65534 (Expected =       65534)  [PASS]
# ABUS =     -1 | BBUS =     -1 | outBUS =           1 (Expected =           1)  [PASS]
#
#  test results summary :
#
#  Total Tests Executed : 9
#  Passed               : 9
#  Failed               : 0
# ** Note: $stop    : tb_top_postFpga_synth.sv(105)
#    Time: 2040 ns  Iteration: 1  Instance: /tb_top
```



The gate-level simulation successfully evaluated all test cases. As demonstrated in the testbench transcript the module computed the correct product for every combination of signed operands, maintaining a 100% pass rate.

FPGA Synthesis Part E: Timing and Operation Verification

Overview While the mathematical results of the post-synthesis simulation exactly match the pre-synthesis simulation, the timing behavior of the signals differs significantly due to the introduction of physical hardware characteristics.

Analysis of Propagation Delay

In the previous RTL simulations, the circuit was evaluated using idealized logic where signals propagated instantly ($t = 0$ delay) upon a clock transition. However, as visible in the post-synthesis waveform there is now a distinct propagation delay between the clock edge and the stabilization of the data buses.

This delay exists because the simulation now models physical Cyclone IV E hardware. It takes physical time for electrical signals to travel through the silicon transistors (LUTs) and routing matrices defined in the .sdo file. Therefore, while an input transition still dictates an output change, the waveform accurately reflects the nanoseconds it takes for that change to arrive at the final output pins.

Verification of Data Capture

Despite the presence of transient states and routing delays, the multiplier reliably captures and outputs the correct data. This robustness is a direct result of proper testbench timing synchronization relative to the hardware speed:

1. **Clock Setup:** The system operates on a 20 ns clock period (forever #10 clk = ~clk;). The physical propagation delay through the FPGA's combinational logic is only a few nanoseconds, comfortably satisfying the setup and hold time requirements within this 20 ns window.
2. **Synchronous Capture:** The testbench is programmed to wait until the controller asserts ready == 1'b1. It then waits for the subsequent negative clock edge (@(negedge clk)) to evaluate outBUS. By capturing on the negative edge, the

testbench ensures it reads the data long after the signals have propagated and stabilized, guaranteeing an accurate assessment of the expected product.